

Entwicklerdokumentation

Projekt: Smart Restaurant Adminsoftware

Sprache: Java 17 (OpenJDK)

Architektur: 3-Schichten (Swing GUI, DAO, SQLite)

Stand: 24. September 2026

Repository: niki356565rt/LF10-Projekt

Status: Fertiggestellt & Geprüft

1. Systemüberblick & Architektur

Die Anwendung verwaltet Mitarbeiter, Speisekarten, Tische und Bestellungen für einen Gastronomiebetrieb. Sie ist als schlanke Desktop-Anwendung ohne externe Webserver oder ORM-Frameworks aufgebaut. Die Datenhaltung erfolgt lokal in einer SQLite-Datenbank über JDBC mit Standard-SQL.

Paket- und Dateistruktur

com.smartrestaurant	
├── Main.java	Anwendungseinstieg & Swing-Start
├── AdminGUI.java	Grafische Benutzeroberfläche (JTabbedPane, JTable)
├── DatabaseManager.java	Data Access Object (DAO) & JDBC Standard-SQL
├── Mitarbeiter.java	Modellklasse Mitarbeiter
├── Artikel.java	Modellklasse Speisen & Getränke
├── Tisch.java	Modellklasse Gaststättentisch
├── Bestellung.java	Modellklasse Bestellung
└── DatabaseTest.java	Automatisierte Integrationstests

2. Datenmodell & SQL-Queries

Die Datenbank `smart_restaurant.db` wird beim ersten Start automatisch durch `DatabaseManager.initDatabase()` erzeugt. Sämtliche Abfragen nutzen Anweisungen nach ANSI-SQL Standard.

Tabellendefinitionen (DDL)

```
-- Tabelle Mitarbeiter
CREATE TABLE IF NOT EXISTS mitarbeiter (
    mitarbeiter_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    benutzername TEXT UNIQUE NOT NULL,
    rolle TEXT NOT NULL,
    aktiv INTEGER DEFAULT 1
);

-- Tabelle Tisch
```

```

CREATE TABLE IF NOT EXISTS tisch (
    tisch_id INTEGER PRIMARY KEY AUTOINCREMENT,
    tischnummer INTEGER UNIQUE NOT NULL,
    kapazitaet INTEGER NOT NULL,
    status TEXT NOT NULL DEFAULT 'frei'
);

-- Tabelle Artikel
CREATE TABLE IF NOT EXISTS artikel (
    artikel_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    kategorie TEXT NOT NULL,
    preis REAL NOT NULL,
    aktiv INTEGER DEFAULT 1
);

-- Tabelle Bestellung
CREATE TABLE IF NOT EXISTS bestellung (
    bestellung_id INTEGER PRIMARY KEY AUTOINCREMENT,
    tisch_id INTEGER NOT NULL,
    erstellt_von INTEGER NOT NULL,
    status TEXT NOT NULL DEFAULT 'aufgegeben',
    erstellt_am TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    gesamtpreis REAL DEFAULT 0.0,
    FOREIGN KEY(tisch_id) REFERENCES tisch(tisch_id),
    FOREIGN KEY(erstellt_von) REFERENCES mitarbeiter(mitarbeiter_id)
);

```

3. Kernklassen & Methoden

Klasse	Verantwortung	Wichtige Methoden
DatabaseManager	JDBC-Verbindung, Tabellenaufbau, CRUD-Operationen via SQL.	initDatabase() getAllMitarbeiter() addMitarbeiter() updateBestellstatus()
AdminGUI	Swing-Hauptfenster. Verwaltet 5 Tabs, Event-Listener und Tabellendarstellungen.	configureTable() refreshAllTables() createMitarbeiterPanel()
Mitarbeiter	Datenhaltung Mitarbeiter.	getId(), getName(), getRolle()
Artikel	Datenhaltung Artikel (Speisen/Getränke).	getPreis(), getKategorie()

Klasse	Verantwortung	Wichtige Methoden
Tisch	Datenhaltung Gaststättentische.	getTischnummer(), getStatus()
Bestellung	Datenhaltung Bestellungen.	getStatus(), getGesamtpreis()
DatabaseTest	Testlauf zur Absicherung aller SQL-Zugriffe.	main(), assertCondition()

4. Bauen und Ausführen

Voraussetzungen

Benötigt wird JDK 17 sowie die JDBC-Treiberdatei `lib/sqlite-jdbc.jar` im Projektordner.

Quellcode kompilieren

```
javac -encoding UTF-8 -cp "lib/sqlite-jdbc.jar;src" -d bin
src/com/smartrestaurant/*.java
```

Anwendung starten

```
java -cp "lib/sqlite-jdbc.jar;bin" com.smartrestaurant.Main
```

Integrationstest ausführen

```
java -cp "lib/sqlite-jdbc.jar;bin" com.smartrestaurant.DatabaseTest
```

5. Qualitätssicherung & Testkonzept

Der automatisierte Test `DatabaseTest` prüft alle SQL-Abfragen beim Bauen der Anwendung. Die Tests stellen sicher, dass Hinzufügen, Auslesen und Löschen in der Datenbank sauber funktionieren.